How to do a full install of an OS1 Raspberry Pi

Purpose: This should walk through the install of a new Raspberry Pi 5 and NVME Hat with Raspberry Pi OS (64-bit) 2024-03-15 for an OS1 3D printer.

Physical setup

We are using a Raspberry Pi 5, a Pimoroni NVME base and a Crucial P3 Plus 1TB SSD.

You can follow the tutorial here to install the SSD and hat.

Flashing Raspberry Pi OS

- Download Raspberry Pi Imager
- Connect NVME Drive to computer using external SSD housing
- In Raspberry Pi Imager
    - Select "Raspberry Pi 5" as device type
    - Select "Raspberry Pi OS (other)"
    - Select "Rapsberry Pi OS (Legacy 64-bit)" *Debian Bookworm based*
    - Select NVME as storage location
    - Customize the OS
        - Set hostname (ie. "OS1")
        - Set timezone to your local timezone
        - Set username (pi) and password
        - Set up WiFi credentials
        - Enable SSH
- You may also need an SD card on the first boot before we set the boot order in the EEPROM.

System setup

Configuring SSD

https://www.jeffgeerling.com/blog/2023/nvme-ssd-boot-raspberry-pi-5

Edit `/boot/firmware/config.txt`

- Add to bottom of /boot/firmware/config.txt
    - `dtparam=pciex1`

Edit the EEPROM on the Raspberry Pi 5.

- `sudo rpi-eeprom-config --edit`
- Change the BOOT_ORDER line to the following:

```
BOOT_ORDER=0xf416
PCIE_PROBE=1
```

- Press Ctrl-O, then enter, to write the change to the file.
- Press Ctrl-X to exit nano (the editor).

Setting Clock

To connect to the internet you may have to update the system clock to be accurate. This can be done using the date command

- `date -s '2014-12-25 12:34:56'` (with the accurate date and time)

Connecting to Wifi

Wifi connection can be handled using the Raspberry Pi OS by connecting the Pi to a keyboard, mouse, and monitor. You may need to work with your institution's IT department to obtain a secure connection to the proper network.

Disable wifi power saving mode

Run the following command in the terminal

```
iw wlan0 get power_save
sudo nmcli c modify [name of the wifi network] 802-11-wireless.powersave 2
sudo systemctl restart NetworkManager
iw wlan0 get power_save
```

Note: if you set up WiFi within the Raspberry Pi Imager, the name of the network will be "preconfigured".

Setting up static ethernet address

Terminal:

```
sudo nmcli connection modify "Wired connection 1" \
    ipv4.method manual \
    ipv4.addresses 192.168.0.1/24
sudo nmcli connection down "Wired connection 1"
sudo nmcli connection up "Wired connection 1"
```

Switching to legacy X11

In bookworm, X11 is deprecated in favor of wayland/wayfire. We need to swap back to X11 for proper light engine functionality.

- `sudo raspi-config`
- Select Advanced Options
- Select Wayland
- Select X11

Disable screen blanking

If we do not disable screen blanking, the HDMI output will shutoff to save power. The light engines are not compatible with this behavior. To do this:

- `sudo raspi-config`
- Select Display Options
- Select Screen Blanking
- Select No

Set USB rules

We need to set the USB permissions for several USB devices. To discover the information on these devices, you can use the `lsusb` command with the usb device connected to the pi. This will list the Bus, Device, VendorID:ProductID, and the common name. We just care about the vendor and product ids. We make a udev rule for each usb device that requires it.

`sudo nano /etc/udev/rules.d/99-thorlabspm100.rules`

- add `SUBSYSTEM=="usb",ATTRS{idVendor}=="1313",ATTRS{idProduct}=="8072",GROUP="users",MODE:="0666"`

Reload udev

- `sudo udevadm control --reload-rules`

Fix Journaling

You also need to update the journal config (retention length and segmentation) at /etc/systemd/journald.conf

```
[Journal]
#Storage=auto
#Compress=yes
#Seal=yes
#SplitMode=uid
#SyncIntervalSec=5m
#RateLimitIntervalSec=30s
#RateLimitBurst=10000
#SystemMaxUse=
#SystemKeepFree=
#SystemMaxFileSize=
#SystemMaxFiles=100
#RuntimeMaxUse=
#RuntimeKeepFree=
#RuntimeMaxFileSize=
#RuntimeMaxFiles=100
MaxRetentionSec=1week
MaxFileSec=1day
#ForwardToSyslog=yes
#ForwardToKMsg=no
#ForwardToConsole=no
#ForwardToWall=yes
#TTYPath=/dev/console
```

```
#MaxLevelStore=debug
#MaxLevelSyslog=debug
#MaxLevelKMsg=notice
#MaxLevelConsole=info
#MaxLevelWall=emerg
#LineMax=48K
#ReadKMsg=yes
```

We are changing the *MaxRetentionSec* and *MaxFileSec* sections. For this to work properly, you also need to create the journal logging directory. Without these commands the old journals are not perserved

Downloading Repo

```
git clone https://github.com/3D-Printing-for-Microfluidics/3D_printer_control.git
```

Installing 3d_printer_control software

Setup Virtual Environment

```
cd 3D_printer_control
python -m venv env
source env/bin/activate
mkdir logs
```

Upgrade pip and setuptools

```
pip install pip --upgrade
pip install setuptools --upgrade
```

Install Galil Driver

Instruction on installing the galil driver are found here but the important parts are below. https://www.galil.com/raspberrypi and maybe here https://www.galil.com/sw/pub/all/doc/gclib/html/. Note: a known working copy of the driver is included in the code repository. It can be installed as follows.

Installing CGLib Software

```
sudo apt install ./requirements/galil-release_1_all.deb
sudo apt update
sudo apt install gclib gcapsd
```

Install Python Packages

`pip install -r requirements/dev.txt`

Setup Flask Database

From the `3D_printer_control` directory run the following: `source rpi/flask_db_setup.sh`

Setup Systemd Service

Give the user authorization to access the X server

```
export DISPLAY=:0.0
xhost si:localuser:pi
```

Create the systemd service with `sudo nano /etc/systemd/system/3d-printer-server.service`

Add the following to the file:

```
[Unit]
Description=3D Printer Server
After=network.target

[Service]
Type=simple
WorkingDirectory=/home/pi/3D_printer_control
Environment=FLASK_APP=/home/pi/3D_printer_control/autoapp.py
Environment=XAUTHORITY=/home/pi/.Xauthority
Environment=DISPLAY=:0.0
ExecStart=/home/pi/3D_printer_control/env/bin/python
/home/pi/3D_printer_control/autoapp.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Reload systemd and start the service with:

```
sudo systemctl daemon-reload
sudo systemctl enable 3d-printer-server.service
sudo systemctl start 3d-printer-server.service
```

Setting custom display resolutions for Visitech

Setup xrandr command on start

Make the autostart folder and entry for our new script.

```
mkdir -p ~/.config/autostart
nano ~/.config/autostart/xrandr.desktop
```

Add the following:

```
[Desktop Entry]
Type=Application
Exec=/home/pi/.config/autostart/xrandr.sh
Hidden=false
NoDisplay=false
X-GNOME-Autostart-enabled=true
Name[en_US]=Xrandr
Name=Xrandr
Comment[en_US]=Set custom resolution using xrandr
Comment=Set custom resolution using xrandr
```

Make our script to set custom resolutions.

- `nano ~/.config/autostart/xrandr.sh`
- Add the following:

```
#!/bin/sh
export DISPLAY=:0.0
#xrandr --newmode "2560x1600_30" 132.00 2560 2608 2640 2720 1600 1603 1609 1623 -
hsync -vsync # old config (hsync causes grayscale issues)
xrandr --newmode "2560x1600_30" 132.25 2560 2608 2640 2720 1600 1603 1609 1623
+hsync -vsync # based on https://tomverbeure.github.io/video_timings_calculator
(CVT-RB)
#xrandr --newmode "2560x1600_30" 132.68 2560 2608 2640 2720 1600 1603 1609 1623
+hsync -vsync # based on edid
#xrandr --newmode "2560x1600_30" 132.6 2560 2608 2640 2720 1600 1602 1604 1625
+hsync -vsync # based on DLPC900 timings
xrandr --addmode HDMI-1 2560x1600_30
xrandr --output HDMI-1 --mode 2560x1600_30

#xrandr --newmode "1920x1080_60" 140.00 1920 1968 2000 2080 1080 1083 1089 1126 -
hsync -vsync
xrandr --newmode "1920x1080_60" 148.50 1920 2008 2096 2200 1080 1084 1089 1125
+hsync -vsync
#xrandr --newmode "1920x1080_30" 62.208 1920 1968 2000 2080 1080 1083 1089 1103 -
hsync -vsync
xrandr --addmode HDMI-2 1920x1080_60
xrandr --output HDMI-2 --mode 1920x1080_60

xset s off
xset s noblank
xset -dpms
```

- `chmod +x ~/.config/autostart/xrandr.sh`

You can check the resolutions after a reboot with:

- `export DISPLAY=:0.0`
- `xrandr`